Embedded Real-Time Control, Alberto Morato

Laboratory 1, I^2C: Line sensor and keypad

Last update: 31-10-2025 Hash: 130be12

1 Introduction

Purpose of this experience is to make you confident in interacting with an external device, connected to the microcontroller through a serial bus.

1.1 SX1509

The main device considered here is the sx1509, which provide 16 digital I/O and specific functionality like a keypad engine. Two sx1509 are connected to the microcontroller through I2C bus; both the devices are shares the same i2c line, in this case **i2c1**. For this reason, to the first sx1509, which we are going to call `sx1509_1`, is assigned the *slave address* `0x3E`, and the second one, which we are going to call `sx1509_2` is associated with the *slave address* `0x3F`.

- **sx1509_1** is used to interact with a line sensor (Pololu QTR Reflectance Sensor)
- **sx1509_2** is set as keypad engine and it is used to interact with a 16-key keypad.

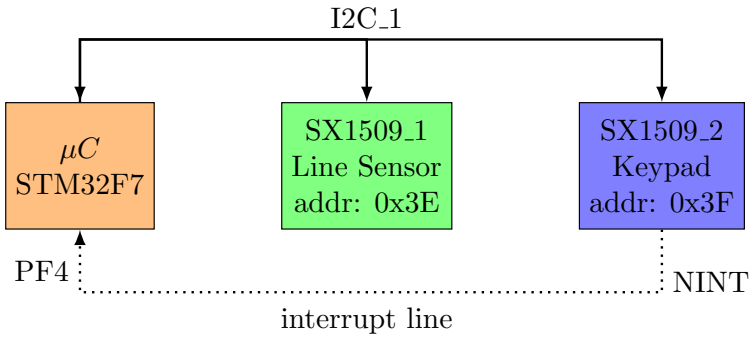


Figura 1: I2C Connection Schema

Sx1509_2 is capable of rising an interrupt if properly configured (see the datasheet for the details); the interrupt pin `NINT` is connected, respectively, to:

- PF4 (GPIO_EXTI4_KPAD_IRQ) for sx1509_2;

Initializing the SX1509 As mentioned above, the SX1509 is a general-purpose I/O expander. To use it as a keypad engine or line sensor, it must be properly configured through initialization. This involves writing specific values to its internal registers over the I^2C interface.

The provided project already includes the code needed to initialize the SX1509 devices, so you do not need to implement this yourself. However, reviewing the code is recommended to better understand how to communicate with an I^2C device.

The initialization code is located in the file `Core/Inc/SX1509_Utils.h`. It defines the function `void Init_SX1509(void)`, which internally calls:

- `void KeypadInit(void)` – to initialize the `sx1509_2` device (used for the keypad),
- `void LineSensorInit(void)` – to initialize the `sx1509_1` device (used for the line sensor).

Additionally, `SX1509_Utils.h` provides useful macros, such as the I^2C addresses of the two devices.

Make sure that:

- `SX1509_Utils.h` is included in your `main.c` file,
- The function `Init_SX1509()` is called **once** in the `main()` function before attempting to use the SX1509 devices.

2 Preliminary operations

2.1 Import an existing project

You will be provided with a project that contains all the code necessary to interact with the sx1509 devices (mainly it deals with the initialization of the peripherals).

1. Download and unzip the project from Moodle.
2. Open STM32CubeIDE
3. File -> Import project from File System. In *Import source* select the folder containing the project. Click *Finish*.

2.2 Enable the interrupts

The STM32 receives an interrupt from the `sx1509_2` (keypad) through pin PF4 each time a key is pressed. In the provided project, this interrupt should already be configured. Check the `*.ioc` file to confirm.

If the interrupt is not enabled, you need to configure pin PF4 as an external interrupt source in the `*.ioc` file. Refer to Laboratory 0 for more details.

Select *External Interrupt Mode with Falling edge trigger detection* and set the pin to *No pull-up and no pull-down*.



Warning: Disable any interrupt for the pin PF2. Ensure that pin PF2 is in **Reset** state.

2.3 Enable support for the printf function

Usually, on PCs, the printf function outputs to the standard output (stdout), which is typically the console we see on a monitor. However, on embedded systems like STM32 where usually we don't have a monitor or a display, you often need to redirect this output to a different medium, such as a debugger console or a serial port.

In order to do that we need to:

1. **Implementing the `_write` Function for `printf` redirection**

   The `_write` function is a low-level system call that allows standard output functions like `printf` to work on embedded systems. By default, `printf` attempts to write to a standard output device (like a terminal), which does not exist on most microcontrollers. To solve this, we redirect the output to a hardware-specific interface.

```
1  int _write(int file, char *ptr, int len)
2  {
3      int DataIdx;
4      for (DataIdx = 0; DataIdx < len; DataIdx++)
5      {
6          ITM_SendChar(*ptr++);
7      }
8      return len;
9  }
```

   In this project, `_write` sends each character to the ITM (Instrumentation Trace Macrocell), a debugging feature available on ARM Cortex-M microcontrollers. ITM allows you to view output in your debugger (such as STM32CubeIDE or Keil) using the SWO (Single Wire Output) viewer.

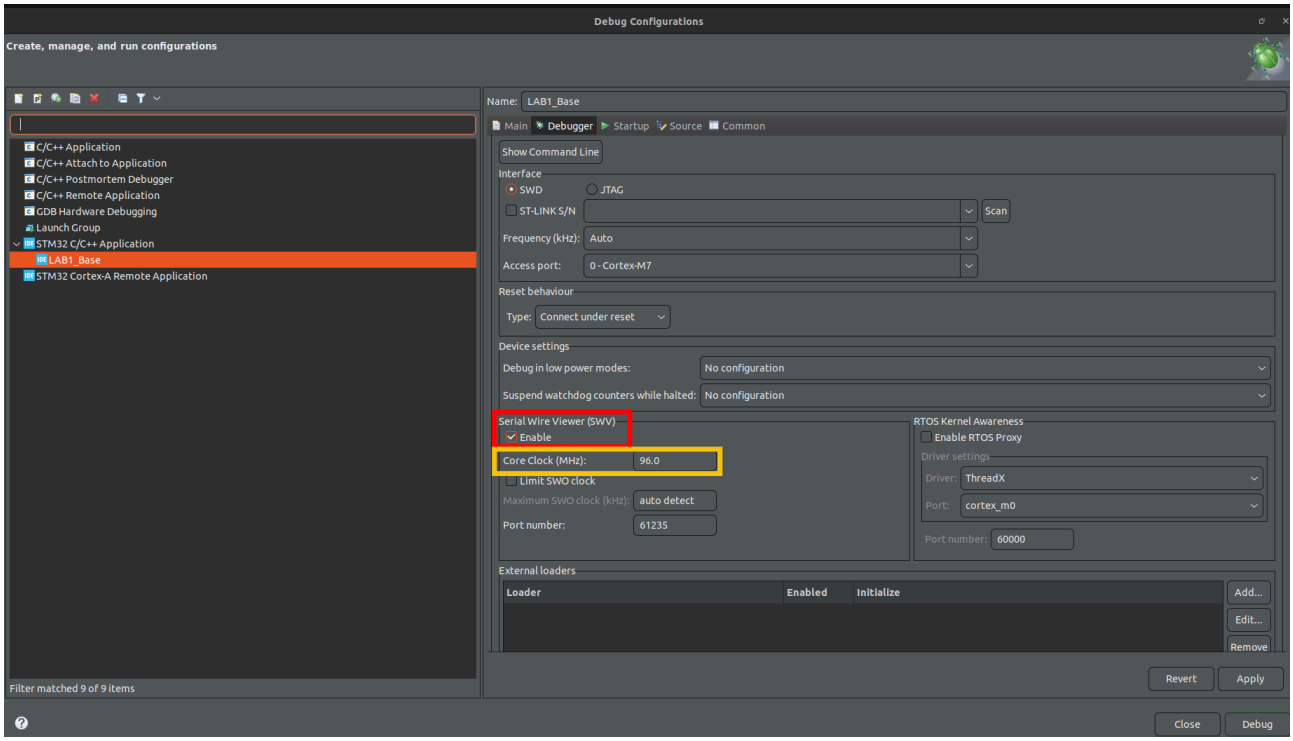
How it works

- The function loops through each character in the output buffer.
- Each character is sent to the ITM using `ITM_SendChar`.
- This enables you to use `printf` in your code and see the output in real time during debugging.

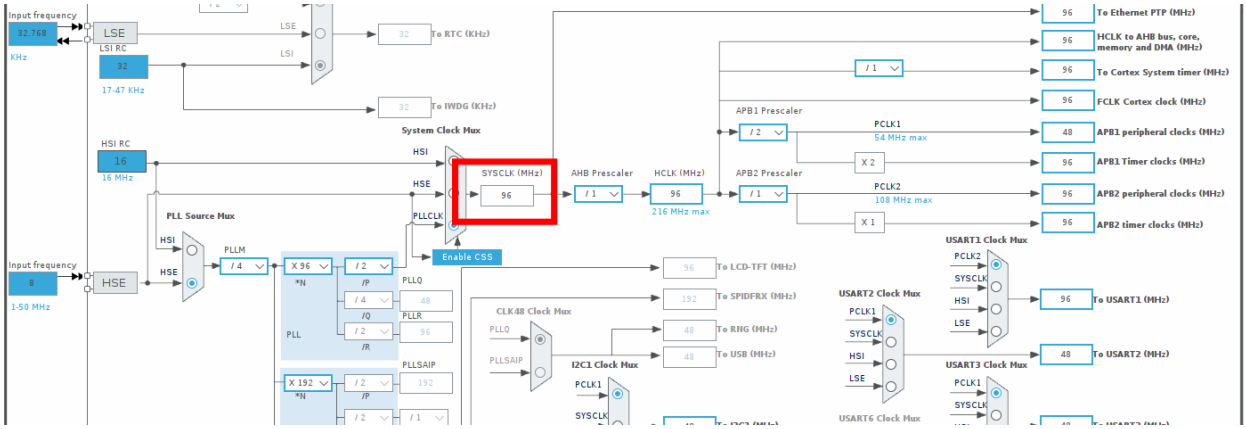
Usage Notes

- In this project, `_write` is already implemented in `Core/inc/print_utils.h`.
- Make sure to include `print_utils.h`, `stdio.h`, and `string.h` in your `main.c` file to use `printf` successfully.

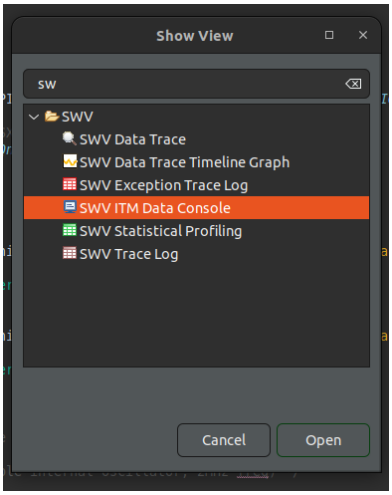
2. Open **Run** > **Debug Configurations**. In the active debug configuration, under the **Debugger** tab, enable **Serial Wire Viewer (SWV)**.



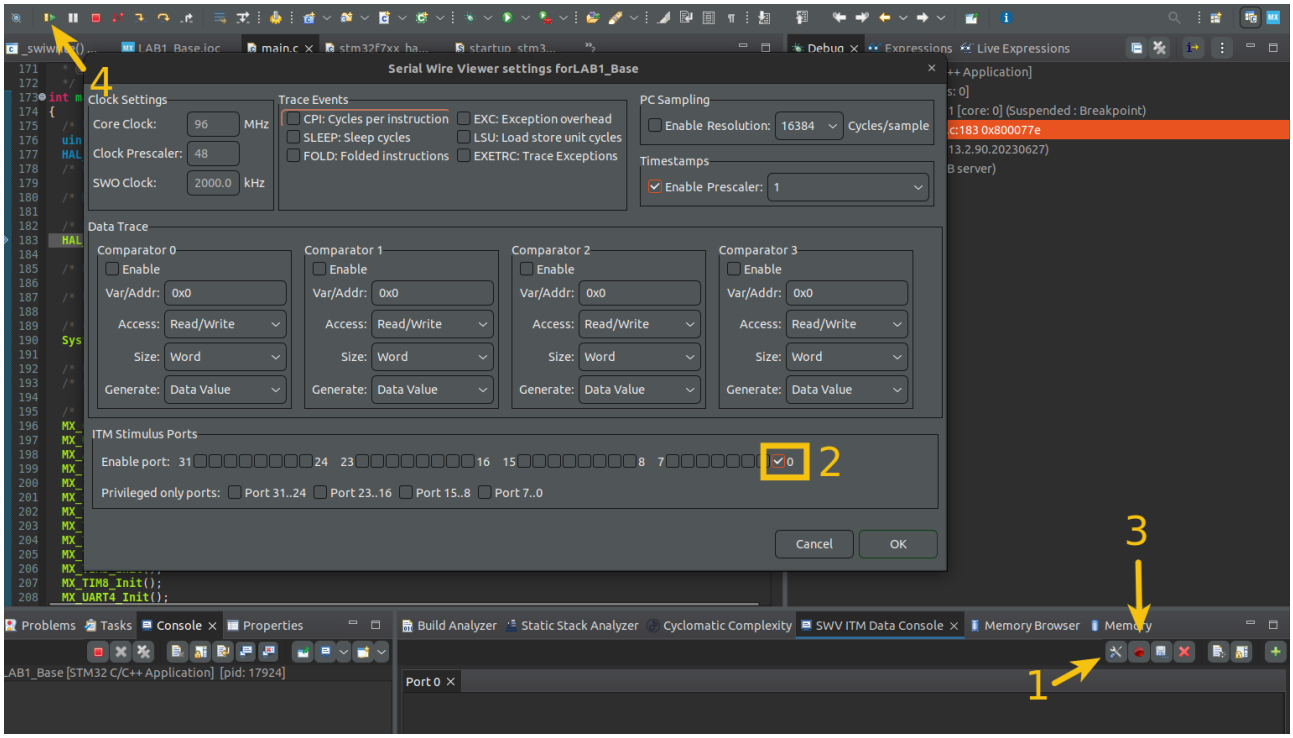
3. Ensure that the **Core Clock** is set to the same frequency as **SYSCLK** in the clock configuration of the `*.ioc` file.



4. Make the ITM log terminal visible by going to **Window** > **Show View** > **SWV** > **SWV ITM Data Console**



5. Start a debug session but without running the code.
6. Find SWV settings and **enable ITM Stimulus Port 0**.



7. Click the red "record" or Start Trace button whenever you want the trace terminal to show you logs.
8. Run the code and you should see the printf output in the SWV ITM Data Console.

2.4 SX1509 Registers

To retrieve or set data from/to the SX1509, you need to read from or write to its internal registers. In addition to the device's I^2C address (see Section 1.1), you must also specify the address of the register you want to access.

The SX1509 includes a range of registers, each with a unique address. Detailed information about these registers can be found in the device's datasheet. For convenience, commonly used register addresses are already defined in the file `SX1509_Registers.h`, so you can refer to them directly in your code without manually looking up the values.

Below are some particularly useful data registers:

- `0x10`: `REG_DATA_B` — holds the data from the line sensor.
- `0x27`: `REG_KEY_DATA_1` — contains the status of the keypad columns.
- `0x28`: `REG_KEY_DATA_2` — contains the status of the keypad rows.

2.5 Keypad

The keypad is a 4x4 matrix, where each row is connected to a a set GPIO pins of the sx1509_2 and each column is connected to another set GPIO pin of the same device. The keypad is connected to the sx1509_2, and the status of the keys is stored in the registers `REG_KEY_DATA_1` (columns) and `REG_KEY_DATA_2` (rows).

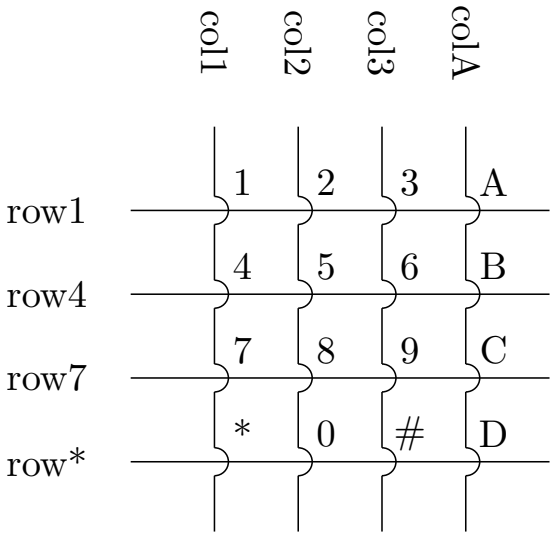


Figura 2: Keypad connection schema

Referring to Figure 2, the LSB (Least Significant Bit) of the `REG_KEY_DATA_1` corresponds to the column 1, the second bit to the column 2, and so on. The same applies to the `REG_KEY_DATA_2` register, where the LSB corresponds to the row 1, the second bit to the row 2, and so on.

Each line of the keypad is connected to a GPIO pin of the sx1509_2 using a pull–up resistor. So the default state of each bit in the GPIO registers will be `1`. In this condition, reading the two registers will give, supposing using an `uint8_t` to store the value, `REG_KEY_DATA_1 = REG_KEY_DATA_2 = 255`.

When a key is pressed, the corresponding row and column will be pulled to ground, so the value of the corresponding bit will be `0`. For example, if the key 1 is pressed, the value of the registers will be

REG_KEY_DATA_1 = 254 and REG_KEY_DATA_2 = 254.

Consider the example in Figure 3, where the key 8 is pressed. Than the value of the registers will be REG_KEY_DATA_1 = 253 and REG_KEY_DATA_2 = 251.

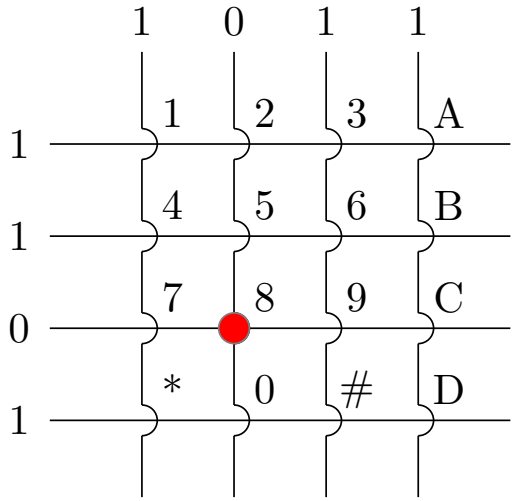


Figura 3: Keypad connection schema with key 8 pressed

2.6 HAL functions

Following are useful HAL functions for this lab. More detailed information [1].

1. `HAL_StatusTypeDef HAL_I2C_Mem_Read(I2C__HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout)`
   Example:

```
HAL_I2C_Mem_Read(
    &hi2c1,                 // Pointer to I2C handle structure (selects which I2C peripheral to use)
    SX1509_I2C_ADDR2 << 1,  // 7-bit device address shifted left by 1 (HAL expects 8-bit format)
    REG_KEY_DATA_1,         // Internal register (memory) address to read from
    1,                      // Size of the memory address (1 = 8-bit register address)
    &data,                  // Pointer to buffer where the read data will be stored
    1,                      // Number of bytes to read
    I2C_TIMEOUT             // Timeout duration for the blocking operation
);
```

2. `HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C__HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout)`

3. `printf(const char *format, ...)`

3 Exercises

3.1 Exercise 1

Write an ISR that recognizes and correctly handles the keypad interrupts. You have to properly implement the `void HAL_GPIO_EXTI_Callback(uint16_t pin)` function. Then print which interrupt has been triggered, i.e., the `pin`. To be able to receive another interrupt from the keypad, have to read the registers REG_KEY_DATA_1 and REG_KEY_DATA_2 inside the ISR.

[1] https://www.st.com/resource/en/user_manual/um1905-description-of-stm32f7-hal-and-lowlayer-drivers-stmicroelectronics.pdf

3.2 Exercise 2

Extend the code of exercise 1 to handle the keypad interrupt. You have to print which keypad button has been pressed.

3.3 Exercise 3

Write a routine that reads the status of the line sensor and prints it. The routine must check the status with a polling period of 100ms.

3.4 Exercise 4

Reimplement exercise 3 using a timer interrupt. Design the appropriate values for the Prescaler and Counter (ARR) that satisfy the requirement.

The configuration involves three main steps within the Device Configuration Tool (`.ioc` file): activating the timer, setting its parameters, and enabling the corresponding interrupt in the Nested Vectored Interrupt Controller (NVIC).

3.4.1 Step 1: Activate the Timer

First, you need to enable the desired timer peripheral. In this example, we use `TIM6`, which is a basic timer without GPIO capabilities, making it ideal for generating time-based events.

- Navigate to the **Categories** panel on the left.
- Expand the **Timers** section.
- Click on **TIM6**.
- In the `Mode` configuration window that appears, check the box for **Activated**.

3.4.2 Step 2: Configure Timer Parameters

After activating the timer, you must configure its timing parameters. These settings determine how often the interrupt will be generated.

- In the `TIM6 Mode and Configuration` panel, go to the **Parameter Settings** tab.
- The two main parameters are **Prescaler (PSC)** and **Counter Period (AutoReload Register)**.
  - **Prescaler (PSC - 16 bits value)**: This value divides the main timer clock. A value of 0 means the clock is divided by 1.
  - **Counter Period (AutoReload Register)**: This is the value the timer counts up to before it overflows and generates an update event, which triggers the interrupt.

The interrupt frequency is calculated with the following formula:

$$\text{Interrupt Frequency} = \frac{\text{Timer Clock Frequency}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

3.4.3 Step 3: Enable the Interrupt in the NVIC

Finally, the interrupt itself must be enabled in the microcontroller's Nested Vectored Interrupt Controller (NVIC) so that the CPU will respond to it.

- In the `TIM6 Mode and Configuration` panel, switch to the **NVIC Settings** tab.
- Check the box next to **TIM6 global interrupt, DAC1 and DAC2 underrun error interrupts**. This makes the timer update interrupt active.

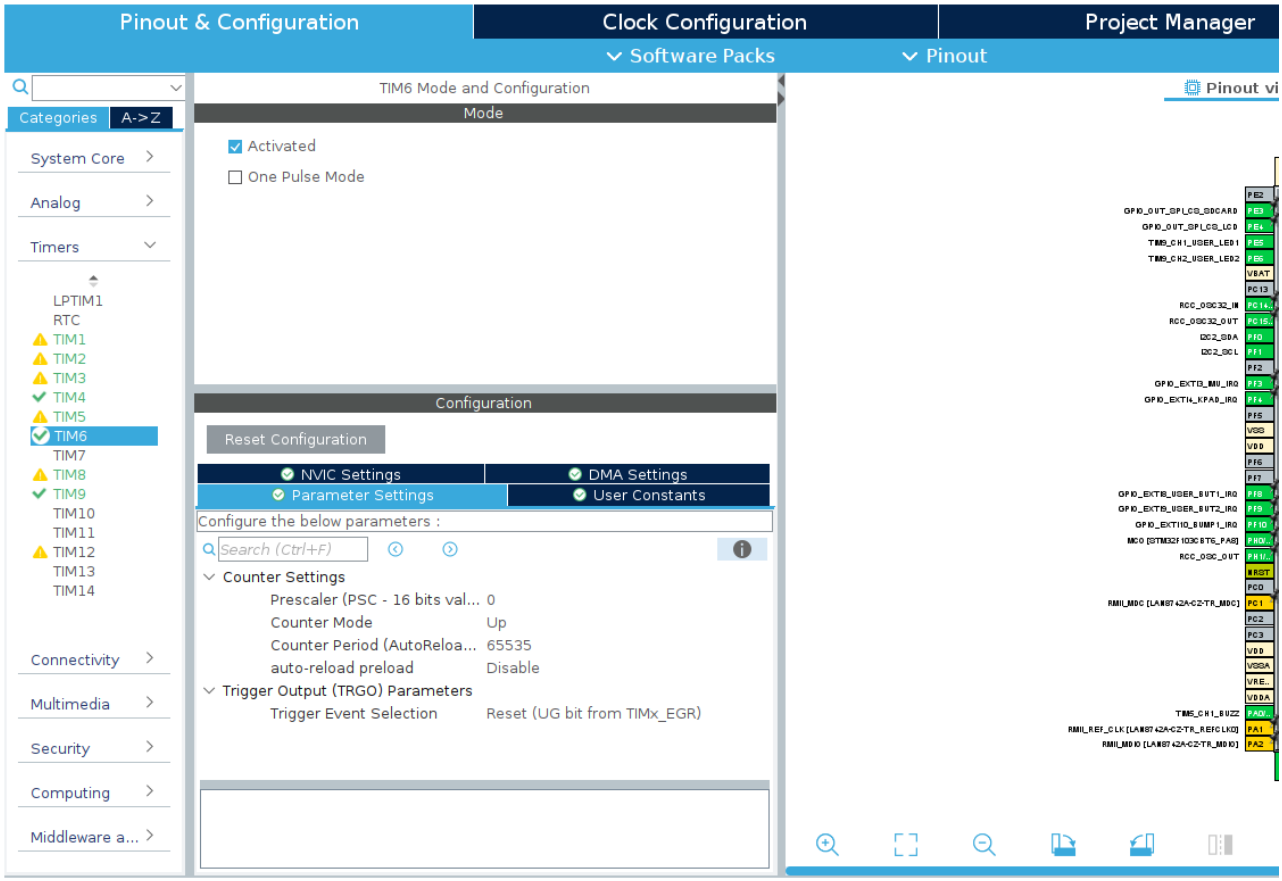


Figura 4: Timer configuration in STM32CubeMX

- You can leave the **Preemption Priority** and **Sub Priority** at their default values (e.g., 0) for this basic example.

3.4.4 Code Implementation

After configuring the timer in the '.ioc' file, save it to generate the initialization code. Then, in your 'main.c' file, you need to add the application logic.

Start the Timer The timer interrupt must be started in your code. This is typically done once in the user code section of the 'main()' function after all peripherals are initialized.

```
1 HAL_TIM_Base_Start_IT(&htim6);
```

Implement the Interrupt Callback Function The STM32 HAL provides a weak-aliased callback function that is called whenever any timer period elapses. You must implement this function in your code to define what happens when the interrupt occurs.

```
1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3   // Check if the interrupt source is our timer (TIM6)
4   if (htim->Instance == TIM6)
5   {
6     // Your interrupt code goes here.
7   }
8 }
```

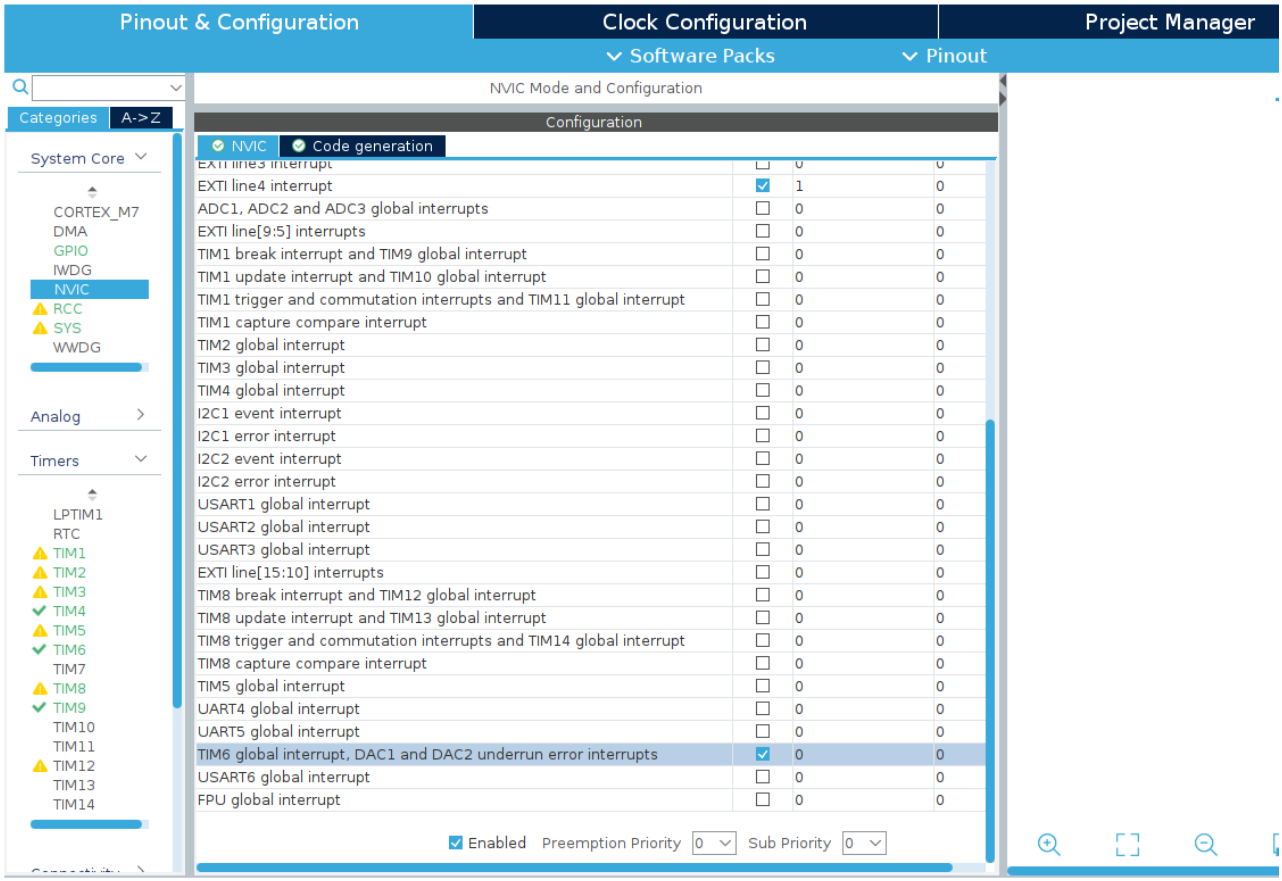


Figura 5: NVIC settings for TIM6 interrupt

3.5 Exercise 5

Extend the code of LAB 0. Make one of the LED blink. If you use LEDs connected to PE5 or PE6, check the `*.ioc` to make sure that those pins are set as `GPIO_output`. The blinking frequency should be set by the user through the keypad. There's two ways to do this:

1. Easy way: make a static mapping between the keypad buttons and the blinking frequency. For example, if the user presses the button 1, the LED should blink with a frequency of 1Hz. If the user presses the button 2, the LED should blink with a frequency of 2Hz, and so on. The mapping is up to you.

2. Hard way (Bonus): The frequency can be set "dynamically" by the user. For example, if the user presses 125#, the LED should blink with a frequency of 125Hz. If the user presses 250#, the LED should blink with a frequency of 250Hz, and so on.